

Spring MVC & REST

A Beginner-Friendly Guide

Sub-Chapter 03-04

1. Spring MVC Architecture	DispatcherServlet, HandlerMapping, ViewResolver
2. @Controller vs @RestController	When to use which annotation
3. Request Mapping Annotations	@GetMapping, @PostMapping, @PutMapping, @DeleteMapping
4. Parameter Annotations	@PathVariable, @RequestParam, @RequestBody, @ResponseBody
5. ResponseEntity and HTTP Codes	Building rich HTTP responses
6. Exception Handling	@ExceptionHandler and @ControllerAdvice
7. REST Best Practices	Versioning, naming, status codes

1 Spring MVC Architecture

DispatcherServlet - HandlerMapping - ViewResolver

Spring MVC is a web framework built on the Front Controller pattern. Every HTTP request passes through a single entry point called the **DispatcherServlet**, which delegates work to the right controller and finally renders a response.

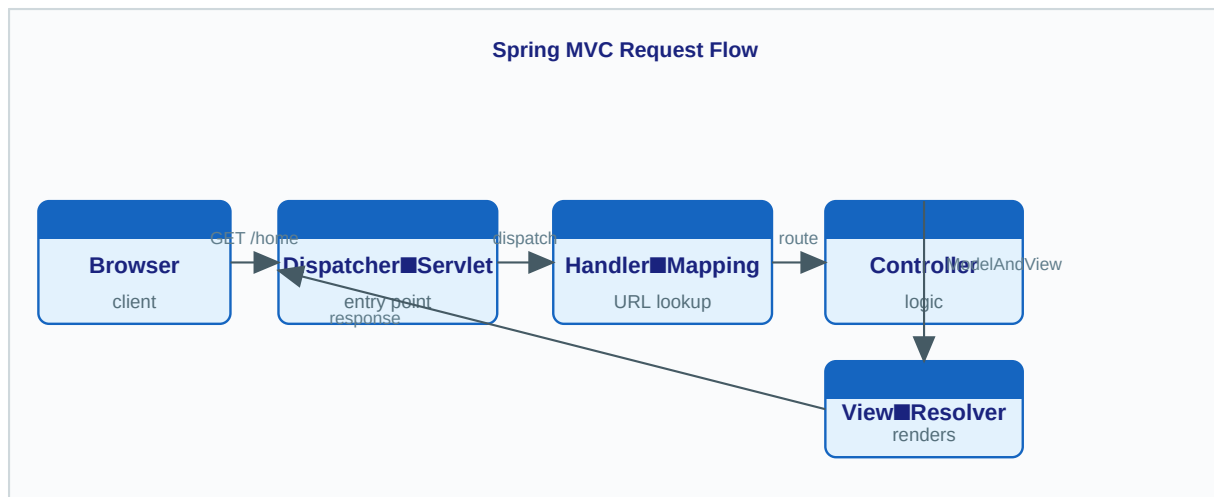


Fig 1.1 - DispatcherServlet routes each request through HandlerMapping to the Controller, then the ViewResolver renders the final response.

Simple Java Example

```
@Configuration
@EnableWebMvc
public class WebConfig implements WebMvcConfigurer {
    @Bean
    public InternalResourceViewResolver resolver() {
        InternalResourceViewResolver r =
            new InternalResourceViewResolver();
        r.setPrefix("/WEB-INF/views/");
        r.setSuffix(".jsp");
        return r;
    }
}
```

Key Rules

- DispatcherServlet is the single entry point for all HTTP requests.
- HandlerMapping maps URLs to controller methods.
- ViewResolver translates logical view names to actual view files.
- Spring Boot auto-configures DispatcherServlet -- no XML needed.
- Use `@EnableWebMvc` to activate Spring MVC configuration manually.

Q1. What is the role of DispatcherServlet?

It acts as the Front Controller, receiving all HTTP requests and delegating them to the right handler.

Q2. What does HandlerMapping do?

It maps incoming request URLs to the correct controller method.

Q3. What is a ViewResolver?

It resolves a logical view name (e.g. home) to a physical view file (e.g. /WEB-INF/views/home.jsp).

Q4. Can you have multiple DispatcherServlets?

Yes, but typically one per application is sufficient. Spring Boot registers one automatically.

Q5. What is ModelAndView?

An object that holds both the model data (Map) and the view name returned by a controller method.

2 @Controller vs @RestController

Choosing the right stereotype annotation

@Controller marks a class as a Spring MVC controller that returns view names. **@RestController** combines **@Controller** and **@ResponseBody** so every method serialises its return value directly to the HTTP response body (JSON/XML).

@Controller Example

```
@Controller
public class PageController {
    @GetMapping("/home")
    public String home(Model model) {
        model.addAttribute("msg", "Hello!");
        return "home"; // resolves to home.jsp
    }
}
```

@RestController Example

```
@RestController
@RequestMapping("/api")
public class ApiController {
    @GetMapping("/greet")
    public String greet() {
        return "Hello, World!"; // written to response body
    }
}
```

Key Rules

- **@Controller** is for traditional MVC applications that render HTML views.
- **@RestController** is for REST APIs that return JSON or XML.
- **@RestController** is a shortcut for **@Controller** + **@ResponseBody**.
- You can still return **ResponseEntity** from a **@RestController** for full control.
- Mixing view rendering and REST in one class is possible but not recommended.

Q1. What is the difference between @Controller and @RestController?

@Controller returns view names; **@RestController** writes the return value directly to the HTTP body.

Q2. What annotation does @RestController combine?

It combines **@Controller** and **@ResponseBody** into a single convenience annotation.

Q3. When should you use @Controller?

When building server-side rendered web pages with Thymeleaf, JSP, or similar template engines.

Q4. Can @Controller return JSON?

Yes, by adding **@ResponseBody** to each method or by returning **ResponseEntity**.

Q5. Does @RestController support path prefixes?

Yes, use **@RequestMapping** at class level to apply a common URL prefix to all methods.

3 Request Mapping Annotations

@GetMapping - @PostMapping - @PutMapping - @DeleteMapping

Spring provides shorthand annotations for each HTTP method. They are composed annotations built on top of @RequestMapping(method=...).

Simple Java Example

```
@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping          // GET /products
    public List<Product> list() { return service.findAll(); }

    @PostMapping         // POST /products
    public Product create(@RequestBody Product p) {
        return service.save(p);
    }

    @PutMapping("/{id}") // PUT /products/{id}
    public Product update(@PathVariable Long id,
        @RequestBody Product p) {
        return service.update(id, p);
    }

    @DeleteMapping("/{id}") // DELETE /products/{id}
    public void delete(@PathVariable Long id) {
        service.delete(id);
    }
}
```

HTTP Method Cheat Sheet

Annotation	HTTP Method	Typical URL	Purpose
@GetMapping	GET	/resources or /resources/{id}	Read one or all
@PostMapping	POST	/resources	Create new resource
@PutMapping	PUT	/resources/{id}	Full update
@PatchMapping	PATCH	/resources/{id}	Partial update
@DeleteMapping	DELETE	/resources/{id}	Delete resource

Key Rules

- @GetMapping is equivalent to @RequestMapping(method = RequestMethod.GET).
- Always map POST to resource creation, GET to retrieval.
- Use PUT for full replacement and PATCH for partial updates.
- DELETE should be idempotent -- calling it twice has the same effect.
- Apply @RequestMapping at class level to share a URL prefix.

Q1. What is @GetMapping a shortcut for?

@RequestMapping(method = RequestMethod.GET) -- it maps HTTP GET requests to a handler method.

Q2. When do you use @PostMapping?

When creating a new resource; the data is sent in the request body as JSON.

Q3. What is the difference between PUT and PATCH?

PUT replaces the entire resource; PATCH applies a partial update to specific fields only.

Q4. Can you map multiple URLs to one method?

Yes: @GetMapping({"/path1", "/path2"}) maps both URLs to the same method.

Q5. What does @RequestMapping at class level do?

It sets a base URL prefix applied to every method mapping inside that controller.

4 Parameter Annotations

@PathVariable - @RequestParam - @RequestBody - @ResponseBody

Spring binds HTTP request data to method parameters using these annotations. Understanding each one is essential for building clean REST endpoints.

Simple Java Example

```
@RestController
@RequestMapping("/items")
public class ItemController {

    // GET /items/42
    @GetMapping("/{id}")
    public Item get(@PathVariable Long id) {
        return service.findById(id);
    }

    // GET /items?page=2&size=10
    @GetMapping
    public List<Item> list(
        @RequestParam(defaultValue="0") int page,
        @RequestParam(defaultValue="10") int size) {
        return service.findAll(page, size);
    }

    // POST /items (body: JSON)
    @PostMapping
    public Item create(@RequestBody Item item) {
        return service.save(item);
    }
}
```

Key Rules

- @PathVariable extracts values from URI template variables like {id}.
- @RequestParam reads query string parameters; supports defaultValue and required.
- @RequestBody deserialises the request JSON/XML body into a Java object.
- @ResponseBody serialises the return value to JSON/XML (already included in @RestController).
- Use @Valid with @RequestBody to trigger Bean Validation automatically.

Q1. What does @PathVariable do?

It binds a URI template variable (e.g. {id} in /items/{id}) to a method parameter.

Q2. How is @RequestParam different from @PathVariable?

@RequestParam reads query string parameters (?key=value); @PathVariable reads URI path segments.

Q3. What does @RequestBody do?

It reads the HTTP request body and deserialises it (e.g. from JSON) into a Java object.

Q4. When is @ResponseBody needed?

On @Controller methods that should write directly to the response body instead of returning a view name.

Q5. How do you make a @RequestParam optional?

Set required=false or provide a defaultValue: @RequestParam(defaultValue="0") int page.

5 ResponseEntity and HTTP Status Codes

Full control over HTTP responses

ResponseEntity lets you control the status code, headers, and body of an HTTP response in a single return value. It is the most flexible way to build REST responses.

Simple Java Example

```
@RestController
@RequestMapping("/users")
public class UserController {

    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        User user = service.findById(id);
        if (user == null) {
            return ResponseEntity.notFound().build(); // 404
        }
        return ResponseEntity.ok(user); // 200
    }

    @PostMapping
    public ResponseEntity<User> create(@RequestBody User u) {
        User saved = service.save(u);
        return ResponseEntity.status(201).body(saved); // 201
    }
}
```

Common HTTP Status Codes

Code	Name	When to Use
200	OK	Successful GET, PUT, PATCH
201	Created	Successful POST (resource created)
204	No Content	Successful DELETE (no body returned)
400	Bad Request	Invalid input or validation failure
401	Unauthorized	Missing or invalid authentication
403	Forbidden	Authenticated but not authorised
404	Not Found	Resource does not exist
500	Internal Server Error	Unhandled server-side exception

Key Rules

- Use `ResponseEntity.ok(body)` for 200, `.status(201).body(body)` for created resources.
- `ResponseEntity.noContent().build()` returns 204 with no body.
- Always return a meaningful status code -- avoid returning 200 for errors.
- Add Location header on 201 responses to point to the new resource URL.
- Use `HttpStatus` enum constants for readability: `HttpStatus.NOT_FOUND`.

Q1. What is ResponseEntity?

A Spring class that represents an HTTP response including status code, headers, and body.

Q2. Which status code should a POST return on success?

201 Created, ideally with a Location header pointing to the newly created resource.

Q3. How do you return a 404 using ResponseEntity?

Return `ResponseEntity.notFound().build()` or `ResponseEntity.status(HttpStatus.NOT_FOUND).build()`.

Q4. What does 204 No Content mean?

The request succeeded but there is no body to return, commonly used for DELETE operations.

Q5. Can you set custom headers in ResponseEntity?

Yes: `ResponseEntity.ok().header("X-Custom", "value").body(data)`.

6

Exception Handling

@ExceptionHandler and @ControllerAdvice

Spring MVC provides two main mechanisms for handling exceptions. **@ExceptionHandler** handles exceptions within a single controller. **@ControllerAdvice** (or **@RestControllerAdvice**) handles exceptions globally across all controllers.

Global Exception Handler Example

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<String> handleNotFound(
        ResourceNotFoundException ex) {
        return ResponseEntity.status(404).body(ex.getMessage());
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handleAll(Exception ex) {
        return ResponseEntity.status(500)
            .body("Internal server error");
    }
}
```

Key Rules

- **@ExceptionHandler** inside a controller handles exceptions only for that controller.
- **@ControllerAdvice** applies exception handling globally to all controllers.
- **@RestControllerAdvice** = **@ControllerAdvice** + **@ResponseBody**.
- More specific exception types are matched before broader ones.
- Always return a meaningful error message with the appropriate HTTP status code.

Q1. What is the purpose of @ExceptionHandler?

It designates a method to handle a specific exception type thrown by controller methods.

Q2. What is @ControllerAdvice?

A specialisation of **@Component** that allows **@ExceptionHandler** methods to apply globally.

Q3. What is @RestControllerAdvice?

It combines **@ControllerAdvice** and **@ResponseBody** so handlers automatically serialise their return values.

Q4. How does Spring choose which handler to invoke?

It selects the most specific **@ExceptionHandler** whose declared exception type matches the thrown exception.

Q5. Can you return custom error objects from @ExceptionHandler?

Yes, return any POJO as the body and Spring will serialise it to JSON automatically.

7 REST Best Practices

Versioning - Naming - Status Codes

Following REST conventions makes your API predictable, easy to consume, and maintainable. The rules below are widely accepted across the industry.

REST Endpoint Design Cheat Sheet

Method	URL	Action	Success Code
GET	/api/v1/orders	List all orders	200
GET	/api/v1/orders/{id}	Get one order by ID	200
POST	/api/v1/orders	Create a new order	201
PUT	/api/v1/orders/{id}	Replace entire order	200
PATCH	/api/v1/orders/{id}	Partial update	200
DELETE	/api/v1/orders/{id}	Delete order	204

Versioning Example

```
// URI versioning (most common approach)
@RestController
@RequestMapping("/api/v1/products")
public class ProductV1Controller { ... }

@RestController
@RequestMapping("/api/v2/products")
public class ProductV2Controller { ... }

// Header versioning alternative:
// @GetMapping(headers = "API-Version=2")
```

Key Rules

- Use plural nouns for resource names: /orders, /users, /products.
- Never use verbs in URLs -- prefer GET /orders over GET /getOrders.
- Version your API from day one: /api/v1/... avoids breaking clients later.
- Use query params for filtering and pagination: ?status=open&page;=0&size;=20.
- Always return consistent error response shapes across the entire API.
- Use HTTPS in production -- REST APIs should never be served over plain HTTP.
- Keep responses stateless -- do not store client session data on the server.

Q1. Why should REST resource URLs use nouns not verbs?

The HTTP method (GET, POST, DELETE) already expresses the action; the URL names the resource only.

Q2. What are the main API versioning strategies?

URI versioning (/v1/), request header versioning, and media-type (Accept header) versioning.

Q3. What does stateless mean in REST?

Each request must contain all the information the server needs; no session state is stored server-side.

Q4. When should you use query params vs path variables?

Path variables identify a specific resource (/orders/42); query params filter or paginate collections.

Q5. What is HATEOAS?

Hypermedia As The Engine Of Application State -- responses include links to related actions, making the API self-discoverable.